

תרגיל בית 2:

שאלה 1:

א. הפונקציה הנתונה תוכל לעבוד רק עם מילונים חד-חד ערכיים משום שערכי המפתחות חייבים להיות ייחודיים (כלומר, לא יהיו שני מפתחות זהים). לפיכך, לא יתקבל כקלט מילון בעל שני מפתחות זהים (אם מאתחילם מילון עם שני מפתחות זהים או יותר, ילקח רק אחד מהמפתחות והערך המתאים שלו), ולכן לאחר ההיפוך כל ערכי המילון המוחזר יהיו ייחודיים. יתרה מכך, במידה וקיימים ערכים זהים למספר מפתחות, לאחר ההיפוך, אלה כמובן יהפכו למפתחות ולכן לא יוכלו להתקיים זה לצד זה ויילקח רק הערך (שיהפוך למפתח) האחרון שהוחזר על ידי `d.items()` (שאינו דרך לדעת מהו) ולכן הפעולה לא "תאבד ערכים" רק כאשר צד הערכים גם הוא ייחודי. לכן, הפונקציה מתקיימת רק עם קלט של מילונים חד-חד ערכיים.

כמו כן, הערכים של הפונקציה הנקלטת חייבים להיות מטיפוס `immutable` משום שבפייתון, רק לטיפוסים מובנים שהם `immutable` יש פונקציית `hash`, כלומר, ניתן למפות אותם למקום שייצג גודל קבוע. לפיכך, טיפוסים שניתנים לשינוי (`mutable`), לא ניתן למפות לגודל קבוע. ישנה חשיבות רבה למיפוי מפתחות מילונים לגודל קבוע בזיכרון משום שזה מאפשר גישה מהירה ויעילה לערכי המילון (כנדרש). בשל כך, אובייקטים מובנים המשמשים כמפתחות של מילונים חייבים לקיים תכונה זו ורק טיפוסים שהם `immutable` (לא ניתנים לשינוי) מקיימים זאת. משום שהפעולה הנתונה בשאלה זו הופכת את ערכי המילון למפתחות, בשל הסיבה שציינתי, גם הערכים חייבים להיות מטיפוס `immutable`.

ב. הפונקציה הנתונה - `reverse_dict()` מייצרת מילון חדש שאותו היא מחזירה ולא משנה את מילון הקלט. כפי שניתן לראות בצילום המסך המצורף (בהמשך), הפונקציה לא שינתה את מספר הזיהוי ומיקומו בזיכרון (`id()`) של מילון הקלט, אלא יצרה מילון חדש מתאים שבעל מספר זיהוי ומקום בזיכרון משל עצמו כמובן. הדבר תואם את אופן הפעולה של הפונקציה הנתונה, העוברת על מילון הקלט ומחזירה אחת חדשה (`return {.....}`). ראהי לכך ניתן לראות בצילום המסך:

```
>>> d1 = {'a':1,'b':2,'c':3,'d':4,'e':5}
>>> hex(id(d1))
'0x111ffd510'
>>> d2 = reverse_dict(d1)
>>> hex(id(d2))
'0x111ffd4c8'
>>> d2
{1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}
>>> hex(id(d1))
'0x111ffd510'
>>> d1
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

ד. הפונקציה בסעיף זה אינה עובדת נכון בשל שאנו מבצעים במהלך ריצת פעולה זו איטרציה על מפתחות המילון ובמהלך כל איטרציה מוסיפים מפתח חדש ומוחקים את המפתח של האיטרציה הנוכחית. מפני שהמילון וערכיו אינם סדורים בזיכרון, אזי כאשר מכניסים ערכים ומפתחות למילון, מיקומם נקבע רנדומלית עבורנו ובשל המשך ריצת הלולאה על מפתחות המילון, במידה ומתווספים ערכים לסוף המילון, נעשה עליהם איטרציה שוב. בפועל, דבר זה מתבטא בכך שאנו מבצעים איטרציה יותר פעמים מאשר מספר הערכים שהיו במילון כאשר החלה הפעולה ומכאן יוצא שיש ערכים שיבצעו עליהם "היפוך" פעמיים. חשוב לציין כי ניתן לתקן זאת באמצעות ביצוע איטרציה לא ישירות על אותו המילון אלא על עותק שונה שלו או באמצעות העתקתו לרשימה (באמצעות $\text{list}(d)$) וכדומה.

שאלה 2:

א. ראשית, חשוב לציין כי מספר פעולות הכפל תלוי אך ורק באיבר b ומכאן שרק במספר הביטים שבאיבר זה - m . וזה משום שריצת לולאת ה- while תלויה רק ב- b וגם הכניסה ל- if (שם יש עוד פעולת כפל) תלויה רק ב- b (האם b אי זוגי). כמו כן, אנו יודעים כי b משתנה בשורה המחלקת את b בשתיים ומעגלת מטה והרי כי זוהי פעולה השקולה להזזת הביט האחרון ימינה. לפיכך, b גדול מאפס (תנאי הלולאה) כל עוד "יש לו ביטים" ולכן הלולאה תרוץ כ- m פעמים.

בכל ריצה של הלולאה מתקיימת לפחות פעולת כפל אחת ובשל ש- b גדול מאפס ונכנס ללולאה גם אם רק הביט המשמעותי ביותר מופעל (התרחיש המינימלי) והמספר זוגי, תהיה הרצת לולאה אחת (האחרונה) בה תהיה כניסה ל- if ולכן יתקיימו שתי פעולות כפל. לסיכום, הוכחתי כי המספר המינימלי של פעולות הכפל יהיה $m + 1$.

באופן זהה לחלק הראשון, גם פה מספר פעולות הכפל תלוי אך ורק ב- b ומכאן שבמספר הביטים שבאיבר זה - m . וזה משום שריצת לולאת ה- while תלויה רק ב- b וגם הכניסה ל- if (שם יש עוד פעולת כפל) תלויה רק ב- b (האם b אי זוגי). כמו כן, b משתנה רק כאשר מחלקים אותו בשתיים ומעגלים מטה, כלומר, מזיזים את הביט האחרון ימינה. לפיכך, הלולאה תרוץ כ- m פעמים. כאשר מסתכלים על מספר מקסימלי של פעולות כפל, אנו רואים כי בתרחיש זה, בכל ריצת לולאה, ניכנס ל- if ולפיכך יתבצעו שתי פעולות כפל. תרחיש מקסימלי זה מתרחש כאשר כל הביטים מופעלים. לסיכום, הראיתי כי המספר המקסימלי של פעולות הכפל יהיה $2 * m$.

שאלה 3:

א. ישנים כ-9898 ביטים בייצוג הבינארי של המספר 30^{2017} . חישובתי זאת בשני דרכים. על פי הנוסחא שנלמדה בכיתה:

```
>>> int(log(30**2017, 2))+1  
9898
```

ועל פי מעבר למחרוזת המייצגת בינארי ומדידת אורכה:

```
>>> len(bin(30**2017)[2:])  
9898
```

ד. למספר טבעי N בעל 7 ספרות בבסיס עשר יש מינימום 20 ביטים ומקסימום של 24 ביטים.

למספר טבעי N בעל 7 בספרות בבסיס 16 כאשר הספרה המשמעותית ביותר שווה או גדולה מ-9 יש יש 28 ביטים (המינימום והמקסימום זהים).
המינימום והמקסימום זהים משום שידוע כי הביט ה-28 בעל משמעות של 2^{27} , כלומר, הוא מוסיף 134217728 (בבסיס עשרוני) ולכן למספר בייצוג בינארי בעל 28 ביטים יש ערך עשרוני בין 134217728 לבין 268435455 והרי שלמספר המינימלי התואם את תנאי השאלה ('9000000') יש ערך עשרוני של 150994944 ולמספר המקסימלי ('FFFFFFF') יש ערך עשרוני של 268435455. כלומר, מספר בינארי בעל 28 ביטים יכול לייצג את שני המספרים ולכן כל מספר שתואם את תנאי השאלה בעל 28 ביטים בייצוג בינארי.

ה. למספר עשרוני N שהומר לבינארי והוסף 1 מצד שמאל. תוצאת הפעולה היא: הסכום של N עם 2 בחזקת לוג בבסיס 2 של N פלוס 1. כלומר, המספר ועוד 2 בחזקת מיקום הביט במקום אחד שמאלה.
$$N + 2^{\lfloor \log_2(N) \rfloor + 1}$$

למספר עשרוני N שהומר לבינארי והוספו k פעמים 1 מצד ימין. תוצאת הפעולה היא: הסכום של המספר N כפול 2 בחזקת k ו-2 בחזקת $k-1$ פחות 1 (הכפלה בערך של המיקום אליו הוזז הביטים של N ועוד סכום הביטים שהוספו k - סכום סדרה הנדסית).
$$2^k * (N + 1) - 1$$

למספר עשרוני N שהומר לבינארי והוספו k פעמים 1 מצד שמאל. תוצאת הפעולה היא: הסכום של N עם 2 בחזקת לוג בבסיס 2 של N פלוס 1 כפול 2 בחזקת k פחות 1. כלומר, המספר ועוד 2 בחזקת מיקום הביט הבא כפול סכום סדרה הנדסית של k האיברים הבאים.
$$N + 2^{\lfloor \log_2(N) \rfloor + 1} * (2^k - 1)$$

ו. ידוע כי מספר N בעל d ספרות ובבסיס b שגדול מאחד מתקיים: $b^{d-1} \leq N < b^d$. לפיכך, למספר בעל בסיס עשרוני מתקיים: $10^{d-1} \leq N < 10^d$. נכניס את אגפי האי שוויון ללוגריתם בבסיס c - $\log_c(10^{d-1}) \leq \log_c(N) < \log_c(10^d)$ ועל פי חוקי הלוגריתם: $(d-1) * \log_c(10) \leq \log_c(N) < d * \log_c(10)$. מעתה והילך נסתכל על הצד הימני של האי שוויון - $\log_c(N) < d * \log_c(10)$.

נסתכל על מקרים של האי שוויון. במקרה הראשון, אם $d * \log_c(10)$ שווה לערך העליון השלם (פונקציית "תקרה") של $\log_c(N)$ אזי הערך השלם העליון של שני צידי האי שוויון שווים (מתקיים שוויון). אם $d * \log_c(10)$ גדול מהערך השלם העליון של $\log_c(N)$ אזי כמובן שמתקיים האי שוויון גם כאשר צידי האי שוויון בערך השלם העליון שלהם. לבסוף, אם $d * \log_c(10)$ קטן מהערך השלם העליון של $\log_c(N)$ (אך עדיין גדול ממנו כמובן) אזי שני צידי האי שוויון שווים (מתקיים שוויון).

לפיכך, לאחר הפעלת פונקציית התקרה נקבל - $\lceil \log_c(N) \rceil \leq \lceil d * \log_c(10) \rceil$. והרי כי ידוע לנו ש- $\lceil \log_c(N) \rceil$ שווה למספר הביטים של N בבסיס c (הוכח בתרגול ובשיעור) ולכן לבסוף ניתן להסיק כי מספר הביטים של N בבסיס עשרוני בעל d ספרות הוא לכל היותר $\lceil d * \log_c(10) \rceil$. מש"ל.

ז. כאשר מבצעים פעולת חיבור של שני מספרים בבסיס בינארי באותו אורך n מבצעים את המספר המקסימלי של פעולות החיבור כאשר מלבד פעולות החיבור הרגילות של כל שני ביטים במיקום המתאים בשני המספרים, מחברים גם את השארית מפעולת החיבור הקודמת. בתרחיש החיבור המקסימלי, החיבור של השארית (ה-carry) לוקח מקסימום 2 פעולות חיבור עבור כל ביט במקום המתאים של שני המספרים וחשוב לציין כי לפחות בחיבור של הביט במקום הראשון (חשיבות 2^0) אין עדיין שארית ולכן תהיה פעולת חיבור אחת. ניתן לראות כי תרחיש זה בו יש מספר מקסימלי של פעולות עם שארית מתרחש כאשר כל הביטים מופעלים בשני המספרים. לפיכך, בשל שעל כל ביט מבצעים בתרחיש זה שתי פעולות חיבור מלבד בחיבור הביטים הראשונים (שם מתבצע פעולת חיבור אחת) ניתן לראות כי מספר פעולות החיבור המקסימלי הינו - $2 * n - 1$.

שאלה 4:

ג. על מנת לבדוק כמה מספרים מתחת ל-100000000 מגיעים קודם ל-89 עם הפעלת הפעולה עבור $p=2$, שיניתי זמנית את הפעולה `count_nums_1` כדי שתבדוק הגעה קודם ל-89 (ולא ל-1) והרצתי את הפעולה עם קלט של 100000000 (החזרתי את הפעולה לאחר מכן למבוקש כמובן).

לאחר הרצת הפעולה התקבל כי ישנם 8581146 מספרים מתחת ל-100000000 שמגיעים קודם ל-89. מצורף צילום מסך:

```
>>> count_nums_89(100000000)
8581146|
```

שאלה 5:

ג. ראשית, ניתן לראות כי אמיר ומיכל מפעילים את הפונקציות בצורה זהה כאשר ההבדל ביניהם המוביל לשוני בזמני הריצה הינו הקלט. בעוד אמיר מעביר לפונקציות `lcs_length_1` ו-`lcs_length_2` שתי מחרוזות רנדומליות שונות, מיכל מעבירה שתי מחרוזות רנדומליות זהות.

נשים לב, כי למרות שוני קריטי זה בין שתי ההרצות, זמן הריצה של `lcs_length_2` כמעט זהה. זה בשל האופן בו הפעולה בודקת את אורך תת המחרוזת הקצרה ביותר המשותפת לקלט. פעולה זו מבצעת את אותו מספר פעולות בלי קשר לתוכן המחרוזת אלא רק תלויה באורך המחרוזות. מספר הפעולות תלוי אך ורק באורך המחרוזת מפני שהיא מייצרת רשימה דו מימדית (ממושש בפייתון על ידי רשימות מוכלות ברשימה) המלאה באפסים ומחשבת את תוכן כל תא ברשימה זו על פי שוויון התוים המתאימים למיקום התא ברשימות שהתקבלו כקלט. לפיכך, גם אם תוים במחרוזות הנתונות שווים או לא, מספר זהה של פעולות יתרחשו כאשר אורך ו"רוחב" הרשימות ישתנה על פי אורך הקלט ולפיכך גם מספר התאים הנבדקים. בשתי ההרצות של אמיר ומיכל, הם מפעילים את הפעולה עם קלט של רשימות בעלות אורך זהה (4000), ולכן, בשל הטענות שהצגתי, אכן זמן הריצה של הפעולה `lcs_length_2` כמעט זהה.

בניגוד לכך, זמן הריצה של `lcs_length_1` שונה באופן מהותי בשתי ההרצות של אמיר ומיכל. זה משום ש-`lcs_length_1` פועל בכך שהוא בודק האם קיימת תת מחרוזת זהה בשתי המחרוזות שהתקבלו כקלט באורך k עד שהוא נעצר כאשר אין תת מחרוזת זהה באורך k ו- k רץ מאחד עד לאורך העצירה. עבור כל k , הוא בודק האם קיימת תת מחרוזת באורך k ומפסיק את המעבר על המחרוזות כאשר הוא מוצא תת מחרוזת זהה באורך k ומחזיר ערך "אמת", לעומת זאת, אם הוא לא מוצא תת מחרוזת זהה באורך k , הוא עובר לגמרי על שתי המחרוזות הנתונות.

לפיכך, על מנת שהפעולה תעצור, עליה לעבור לגמרי על שתי המחרוזות רק פעם אחת ובשאר המעברים על המחרוזות לעבור רק על חלק מהמחרוזות (עד שהוא מוצא את תת המחרוזת הזוה באורך k).

בשל כך, כאשר אמיר מריץ את הפעולה עם שתי מחרוזות רנדומליות שונות, הפעולה צריכה לעבור על המחרוזות עד שהיא מוצאת תת מחרוזת באורך k ופעם נוספת על כל איברי המחרוזות כאשר הוא לא ימצא תת מחרוזת באורך k . בשל היותן של המחרוזות רנדומליות שונות אזי k יהיה קטן מאוד ביחס לאורך המחרוזות (4000) ולכן מלבד האיטרציה האחרונה, שאר האיטרציות יהיו קצרות (בהתאם ל- k) ובטח שרובן לפחות לא יתקרבו בכלל למעבר על כל תוי המחרוזות (הסיכוי אפסי שכן).

לעומת זאת, כאשר מיכל מריצה את הפעולה עם שתי מחרוזות רנדומליות שונות, הפעולה תחפש תת מחרוזת זהה מאורך k ועד לאורך המחרוזת (4000 במקרה שלנו) ולכן ולכן תעבור על המחרוזות כ-4000 פעמים כאשר ככל ש- k גדול יותר, היא צריכה לעבור על יותר איברים במחרוזות (כ- k איברים) וזה משום שהמחרוזות הרי זהות.

לסיכום, ניתן לראות כי בשל הטענות שהצגתי זמן הריצה של lcs_length_1 תלוי ישירות בתוכן המחרוזות. כמו כן, זמן הריצה המקסימלי של פעולה זו יתקבל כאשר המחרוזות הנתונות יהיו זהות בשל הטענות שהצגתי. עובדה זו תואמת את זמני הריצה של ההרצות של אמיר ומיכל, שכן אמיר הריץ את הפעולה עם מחרוזות רנדומליות שונות ואילו מיכל הריצה אותה עם מחרוזות זהות והרי שקיבלה זמן ריצה גדול בהרבה מאשר של אמיר.